BÖLÜM 7 / MODULE 7

AI Ajanları (AI Agents), Otonom Akışlar ve Bellek Mimarisi

AI Agents, Autonomous Loops & Conversational Memory Systems

LLM vs AI Agent farkı, ReAct (Reasoning + Acting) döngüsü, kısa ve uzun vadeli bellek (ChatMemory vs Vector Memory), Plan-and-Solve, Reflexion, Çoklu Ajan (Multi-Agent Supervisor) ve LangChain4j durumsal ajan tasarımı.

LLM vs AI Agent taxonomy, ReAct iterative loop, short/long-term memory architectures (Sliding Window ChatMemory vs Vector Memory), Plan-and-Solve, Reflexion self-correction, Multi-Agent Supervisor networks, and stateful orchestration in Java.

37

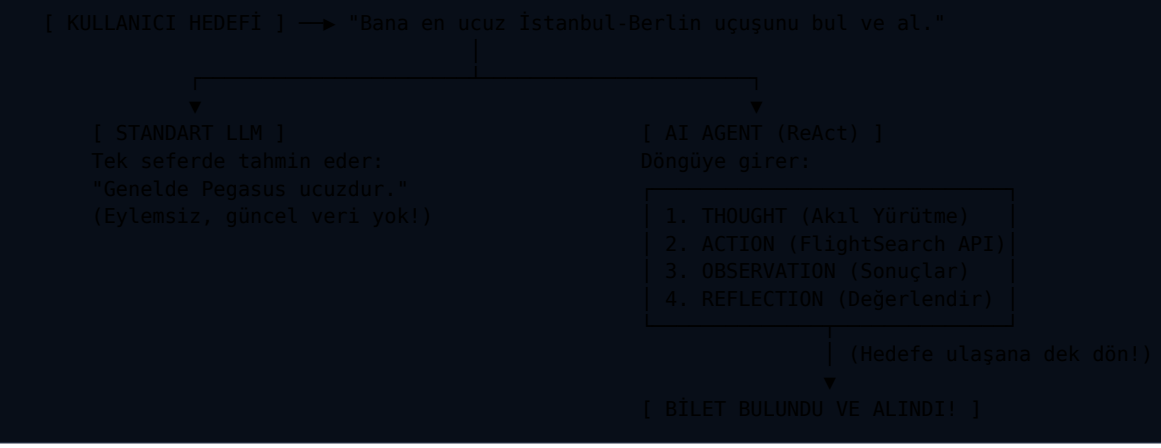
BÖLÜM: AI AJANLARI (AI AGENTS), OTONOM AKIŞLAR VE BELLEK MIMARISI | MODULE: AI AGENTS, AUTONOMOUS LOOPS & CONVERSATIONAL MEMORY SYSTEMS

LLM vs AI Agent: ReAct (Reasoning + Acting) Döngüsünün Anatomisi

LLM vs AI Agent: The Autonomous ReAct (Reasoning + Acting) Loop

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Normal bir LLM, kütüphanede oturan çok bilmiş bir profesör gibidir: Ona ne sorarsanız konuşur, anlatır ama yerinden kalkıp bir bardak su bile getiremez (Sadece Konuşur). Bir AI Agent ise sırtına çanta takıp sokağa fırlayan bir özel dedektiftir! Bir hedef verirsiniz; dedektif önce düşünür (Thought: 'Nereden başlamalıyım?'), sonra harekete geçer (Action: 'Havaalanına git'), sonra gördüklerini inceler (Observation: 'Uçak kalkmış') ve hedefine ulaşana kadar bu döngüyü tekrarlar!

🇬🇧 ENGLISH (ELI5)

A standard LLM is a brilliant scholar seated in an armchair: ask anything, and they will lecture eloquently, but they physically cannot leave the room to brew a cup of coffee (Output is purely tokens). An AI Agent is a field operative! Given a high-level goal, it autonomously drives a ReAct loop: it reasons about its current state (Thought), interacts with the environment (Action: invoking tools), ingests the output (Observation), and iterates until the objective is satisfied!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ReAct (Yao et al., Princeton/Google) Çerçevesinin Mühendisliği: 1) Thought: Model içsel bir akıl yürütme adımı atar ('Kullanıcının bütçesini kontrol etmeliyim'). 2) Action: Hangi tool'un hangi parametrelerle çağrılacağına karar verir (`checkBudget(userId)`). 3) Observation: Aracın ürettiği ham sonuç prompt'a eklenir. 4) Loop Termination: Model hedefin tamamlandığını anladığında nihai yanıtı döner veya durma kriterine (Max Iterations, ör. 10 adım) takılır. 5) Sonsuz Döngü Koruması: Ajanlar aynı hatayı tekrar tekrar yaparak sonsuz döngüye girebilir; bu yüzden sıkı bir `maxIterations` koruması zorunludur.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ReAct Framework & Runtime Controls: 1) Cognitive Loop: Interweaves reasoning traces with task-specific actions. The thought token stream allows the model to track progress, handle exceptions, and dynamically adjust action plans. 2) Termination Condition: The cycle terminates when the model produces an answer token stream instead of a tool invocation. 3) Infinite Loop Mitigation: Autonomous agents frequently enter pathological loops (repeatedly calling failed tools). Systems must enforce a strict `max_steps` threshold (typically 5 to 10), throwing an escalation event if exceeded.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: ReAct Ajan Döngüsü ve Adım Kontrolcüsü (Loop Governor)
public class ReActAgentGovernor {
    private final int maxSteps;

    public record AgentStep(int stepIndex, String thought, String action, String observation) {}

    public ReActAgentGovernor(int maxSteps) {
        this.maxSteps = maxSteps;
    }

    public boolean canContinue(int currentStep) {
        if (currentStep >= maxSteps) {
            IO.println("⚠ AJAN GÜVENLİK SINIRI: Maksimum adım sayısına (" + maxSteps + ") ulaşıldı!
Döngü kırıldı.");
            return false;
        }
        return true;
    }
}

void main() {
    var governor = new ReActAgentGovernor(5);
    for (int step = 1; step <= 6; step++) {
        if (!governor.canContinue(step)) break;
        IO.println("Adım #" + step + " yürütülüyor (Thought -> Action -> Observation)");
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Maliyet Uyarısı (Agent Infinite Burn): Sonsuz döngü koruması olmayan bir ajan, bir API hata verdiğinde sürekli aynı isteği atarak dakikalar içinde yüzlerce dolarlık token yakabilir! Her ajanın mutlak bir `maxSteps = 8` limiti ve toplam bütçe tavanı olmalıdır.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Cost hazard: An unconstrained ReAct agent caught in an exception retry loop can execute 50 recursive calls in minutes, burning significant token budget. Always inject hard bounds: `max_iterations \<= 8` and execution timeout gates.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• LLM sadece metin üretir; AI Agent ise hedefe ulaşmak için araçları otonom kullanır.

• ReAct döngüsü: Thought (Düşün) -> Action (Eyleme Geç) -> Observation (Gözlemle) adımlarından oluşur.

• Üretim ortamındaki tüm ajanlar mutlaka maksimum adım ve bütçe sınırıyla korunmalıdır.

🎯 Key Takeaways for the Interview

• An LLM generates text; an AI Agent autonomously reasons, acts, and iterates toward a goal.

• The ReAct loop operates via interleaved Thought, Action, and Observation cycles.

• All production agents must enforce strict maximum step limits and circuit breakers.

38

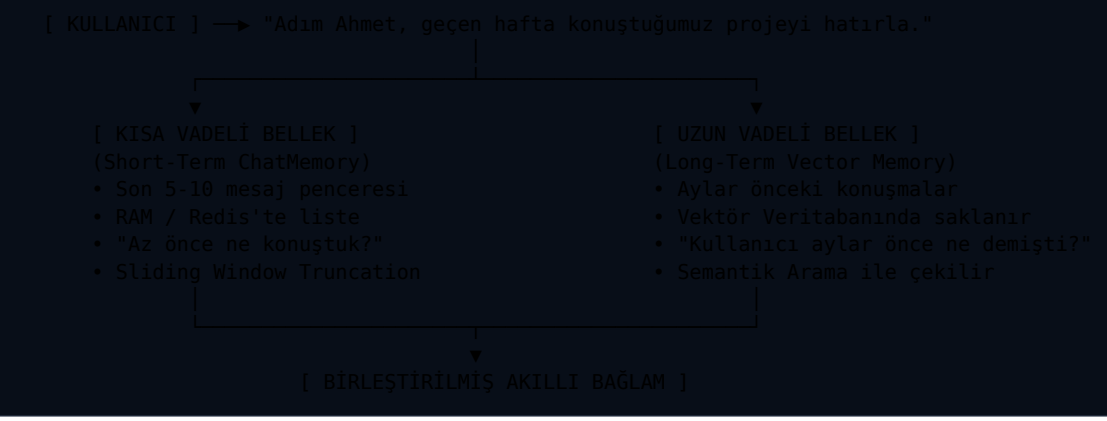
BÖLÜM: AI AJANLARI (AI AGENTS), OTONOM AKIŞLAR VE BELLEK MIMARISI | MODULE: AI AGENTS, AUTONOMOUS LOOPS & CONVERSATIONAL MEMORY SYSTEMS

Kısa ve Uzun Vadeli Bellek: ChatMemory vs Vector Memory

Memory Architectures: Short-Term (Sliding Window ChatMemory) vs Long-Term (Vector Memory)

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

İnsan beynini düşünün: Çalışma belleğiniz (Kısa Vadeli Bellek), şu an bu cümleyi okurken aklınızda tuttuğunuz kelimelerdir. 10 dakika sonra bu kelimeleri unutursunuz ama ana fikri uzun vadeli hafızanıza atarsınız. 1 yıl sonra bir arkadaşınızı gördüğünüzde ise uzun vadeli hafızanızdan 'Bu Ahmet, geçen yaz tatile gitmiştik' diye anıyı geri çağırırsınız (Vektör Belleği). Yapay zekada da son 5 mesajı Redis'te tutmak kısa vadeli bellektir; aylar önceki tercihleri vektör veritabanında saklayıp gerektiğinde aramak uzun vadeli bellektir!

🇬🇧 ENGLISH (ELI5)

Consider human cognition: Short-term working memory holds the specific words you read 10 seconds ago. After the conversation ends, transient words fade, but core facts consolidate into your long-term hippocampal memory. When you meet an acquaintance a year later, your brain executes a semantic memory retrieval: 'Ah, this is Alex from the London conference.' In AI architectures, short-term memory is a fast Redis list holding the last N turns; long-term memory is a vector store indexing lifetime user facts!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Bellek Mimarisi Mühendisliği: 1) Short-Term (Sliding Window ChatMemory): `chatClient` her çağrıldığında son N mesajı (ör. son 10 mesaj) prompt'a ekler. Token sınırı doldukça en eski mesajlar listeden düşer (FIFO). 2) Summary Memory: Eski mesajlar silinmek yerine arka planda küçük bir modelle (GPT-4o-mini) 2 paragraflık bir 'Özet' haline getirilir. 3) Long-Term Vector Memory: Kullanıcının kalıcı tercihleri ('Fıstığa alerjim var', 'TypeScript severim') entity extraction ile yakalanır, vektörleştirilir ve kullanıcı ID'si ile indekslenir. İlgili konu açıldığında RAG ile çekilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Memory Engineering Paradigms: 1) Sliding Window (Short-Term): Buffers the last K conversational turns. As tokens breach a budget threshold, oldest turns are evicted (FIFO). 2) Conversational Summary Buffer: As history lengthens, background workers compress evicted turns into an evolving running summary prepended to the system prompt. 3) Long-Term Episodic Vector Memory: Fact extraction pipelines parse dialogues for user preferences ('I prefer dark mode and Java 25'), project them into a persistent vector store partitioned by `user_id`, and retrieve them on-demand via semantic similarity.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: Sliding Window ChatMemory ve Redis Entegrasyonu
import org.springframework.ai.chat.memory.ChatMemory;
import org.springframework.ai.chat.memory.InMemoryChatMemory;
import org.springframework.ai.chat.messages.Message;
import java.util.List;

public class ConversationMemoryManager {
    private final ChatMemory chatMemory;
    private final int maxMessagesWindow;

    public ConversationMemoryManager(int maxMessagesWindow) {
        this.chatMemory = new InMemoryChatMemory(); // Üretimde RedisChatMemory kullanılır
        this.maxMessagesWindow = maxMessagesWindow;
    }

    public List<Message> getRecentHistory(String conversationId) {
        List<Message> allMessages = chatMemory.get(conversationId, 100);
        if (allMessages.size() <= maxMessagesWindow) {
            return allMessages;
        }
        // Sliding window: Sadece son maxMessagesWindow kadar mesajı al
        return allMessages.subList(allMessages.size() - maxMessagesWindow, allMessages.size());
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Bütçe ve Güvenlik Tuzağı: Kullanıcının 50 mesajlık sohbet geçmişini filtrelemeden olduğu gibi her isteğe eklerseniz, hem her istekte 10.000 token harcarsınız hem de kullanıcının 20 mesaj önce yaptığı bir şaka modeli halüsinasyona sürükleyebilir. Sohbet geçmişi daima token sınırlı bir pencere (Window) ile tutulmalıdır.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Context blowout trap: Appending infinite conversational history without bounds exponentially scales token costs and dilutes model attention. Always enforce strict sliding window limits or automated summarization buffers.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Kısa vadeli bellek (Sliding Window), anlık diyalog akışını Redis/RAM üzerinde tutar.
- Uzun vadeli bellek (Vector Memory), kullanıcı tercihlerini semantik olarak arşivler.
- Eski diyalogları periyodik olarak özetlemek (Summarization), bağlamı ve maliyeti korur.

🎯 Key Takeaways for the Interview

- Short-term memory buffers active conversational turns in low-latency stores.
- Long-term episodic memory archives enduring user preferences in vector stores.
- Periodic conversation summarization preserves context without token explosion.

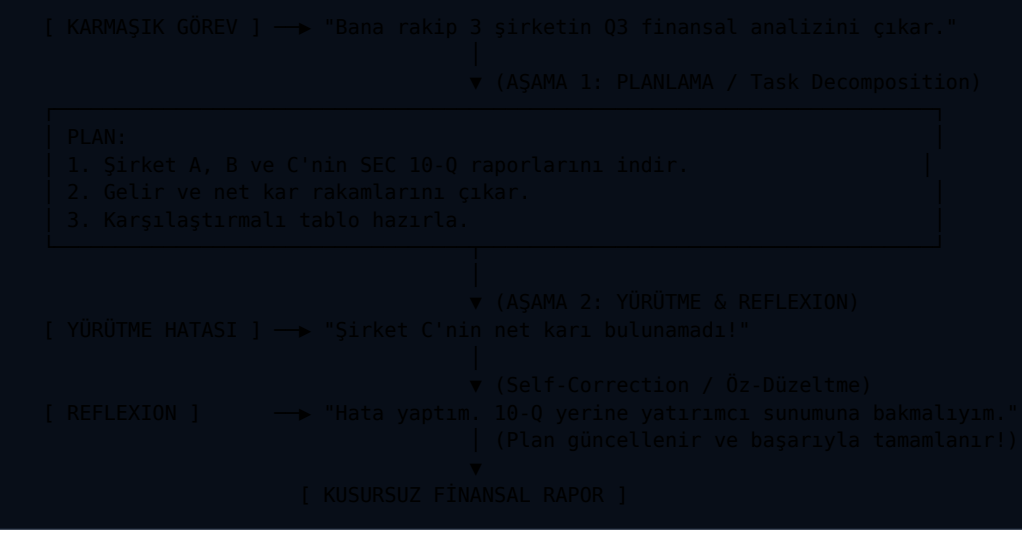
39

BÖLÜM: AI AJANLARI (AI AGENTS), OTONOM AKIŞLAR VE BELLEK MIMARISI | MODULE: AI AGENTS, AUTONOMOUS LOOPS & CONVERSATIONAL MEMORY SYSTEMS

Planlama ve Görev Ayrıştırma: Plan-and-Solve, Reflexion & Self-Correction

Planning & Task Decomposition: Plan-and-Solve, Reflexion & Self-Correction Loops

🤖 AI Mimarisi & Veri Akış Şeması 🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir satranç ustası hemen elini tahtaya uzatıp taş oynamaz. Önce durur, 5 hamle sonrasını kafasında planlar: 'Atı buraya koyarsam, o piyonu sürer, ben de vezirle şah çekerim' (Plan-and-Solve). Eğer rakip beklemediği bir hamle yaparsa 'Eyvah, planım bozuldu, hemen yeni bir strateji kurmalıyım' der ve planını revize eder (Reflexion / Self-Correction). Basit yapay zeka hemen cevaba atlar; akıllı bir yapay zeka ajanı ise önce plan yapar, hata yaparsa hatasından ders çıkarıp kendini düzeltir!

🇬🇧 ENGLISH (ELI5)

A chess grandmaster never reaches blindly across the board. They mentally simulate five moves ahead: 'If I advance my knight, they will counter with a pawn push, opening my bishop for checkmate' (Plan-and-Solve). When an unexpected blunder occurs, they reflect critically, analyze the tactical error, and pivot to a defensive counter-strategy (Reflexion / Self-Correction). Naive LLMs answer impulsively; advanced agents draft a structured multi-step plan, self-critique intermediate outputs, and correct mistakes autonomously!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Planlama Mimarisi Algoritmaları: 1) Plan-and-Solve (Wang et al., 2023): Görevi tek seferde çözmek yerine önce bağımsız alt adımlara (Sub-tasks DAG) böler. Adımlar birbirine bağımlıysa (Dependency Graph) sırayla, bağımsızsa Java 25 Virtual Threads ile paralel yürütülür. 2) Reflexion (Shinn et al., 2023): Ajan her eylemden sonra bir 'Öz-Eleştiri' (Self-Critique) adımı çalıştırır: 'Ürettiğim SQL geçerli mi? Çıktı beklenen şemaya uyuyor mu?' Eğer uymuyorsa hata logunu kendi hafızasına ekleyip eylemi yeniden dener. 3) LLM Compiler: Planlama ile yürütmeyi paralel task scheduler ile optimize eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Advanced Agentic Planning Topologies: 1) Plan-and-Solve Paradigm: Explicitly decouples macro-planning from execution. Synthesizes a Directed Acyclic Graph (DAG) of discrete sub-tasks. Independent DAG nodes execute concurrently across Java 25 Virtual Threads. 2) Reflexion Architecture (Shinn et al.): Augments agents with dynamic memory and self-reflective feedback loops. If code generation fails a unit test, the execution trace is injected back into the prompt scratchpad, triggering targeted self-correction. 3) Deterministic Rollback: State machines track DAG state; failed sub-steps can be retried without restarting the entire pipeline.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Plan-and-Solve Görev Ayrıştırma ve Plan Modeli
import java.util.List;

public class TaskPlannerEngine {
    public record SubTask(int stepOrder, String description, boolean isCompleted) {}
    public record ExecutionPlan(String overallGoal, List<SubTask> subTasks) {}

    public static ExecutionPlan createPlanForQuery(String userGoal) {
        // Modelin ürettiği DAG planı simülasyonu
        var task1 = new SubTask(1, "Postgres veritabanından son 30 günün satışlarını çek", true);
        var task2 = new SubTask(2, "Müşteri iade oranlarını hesapla", true);
        var task3 = new SubTask(3, "Grafik için özet JSON oluştur", false);

        return new ExecutionPlan(userGoal, List.of(task1, task2, task3));
    }
}

void main() {
    var plan = TaskPlannerEngine.createPlanForQuery("Aylık E-Ticaret Sağlık Raporu");
    IO.println("Hedef: " + plan.overallGoal());
    for (var task : plan.subTasks()) {
        IO.println("Adım " + task.stepOrder() + ": " + task.description() + " [Tamamlandı: " +
task.isCompleted() + "]");
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mülakat Uyarısı: Reflexion ve Self-Correction döngülerine de mutlaka `maxRetries = 3` sınırı konulmalıdır. Eğer model 3 denemede de hatayı düzeltemiyorsa inatla devam edip token yakmamalı, hemen 'Hata çözülemedi, insan müdahalesi gerekiyor' diyerek durmalıdır.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Reflexion loop hazard: A model hallucinating an impossible solution will self-reflect into an infinite loop if unconstrained. Enforce strict `max_reflections \<= 3` bounds and fail-fast with human escalation if errors persist.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Plan-and-Solve, karmaşık problemleri yönetilebilir alt görevlere (Sub-tasks) böler.

• Reflexion (Öz-Düzeltme), başarısız olan adımları analiz edip hatadan ders çıkarır.

• Planlama adımları Java 25 sanal iş parçacıklarıyla asenkron ve paralel çalıştırılabilir.

🎯 Key Takeaways for the Interview

• Plan-and-Solve decomposes complex requests into an executable DAG of sub-tasks.

• Reflexion enables agents to learn from intermediate execution errors dynamically.

• Independent plan steps can be scheduled concurrently using Java 25 Virtual Threads.

40

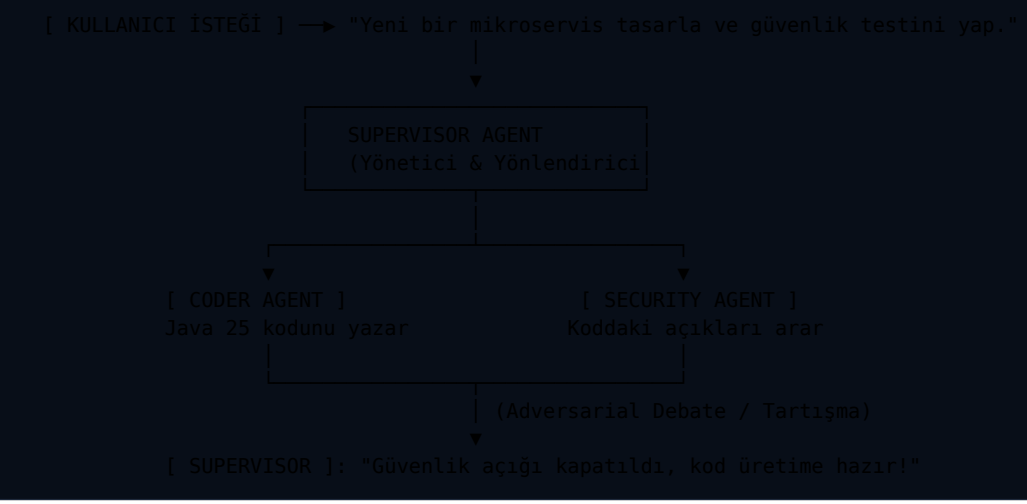
BÖLÜM: AI AJANLARI (AI AGENTS), OTONOM AKIŞLAR VE BELLEK MIMARISI | MODULE: AI AGENTS, AUTONOMOUS LOOPS & CONVERSATIONAL MEMORY SYSTEMS

Çoklu Ajan Sistemleri (Multi-Agent Workflows: Supervisor, Worker & Debate)

Multi-Agent System Architectures: Supervisor-Worker Networks & Adversarial Debate

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirkette tek bir kişinin hem muhasebeye, hem yazılıma, hem de hukuka bakmasını beklerseniz o şirket batar. Her işin ayrı bir uzmanı vardır: Bir yazılımcı kod yazar, bir siber güvenlikçi kodu inceler ve eksikleri söyler, bir yönetici de ikisini koordine eder. Çoklu Ajan (Multi-Agent) mimarisi tam olarak bu ofistir! Tek bir generale tüm işi yüklemek yerine; her biri tek bir konuda uzmanlaşmış küçük yapay zeka ajanları (Coder, Reviewer, Tester) birbiriyle konuşur ve tartışarak mükemmel iş üretir!

🇬🇧 ENGLISH (ELI5)

In a successful software company, you don't hire one person to simultaneously write backend code, audit cryptographic security, and file tax compliance. You establish specialized roles: an Engineer authors code, a Security Auditor attacks it, and an Engineering Manager coordinates the deliverables. Multi-Agent Systems instantiate this exact organizational hierarchy: specialized agents collaborate, challenge each other via debate loops, and deliver hardened results!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Çoklu Ajan Topolojileri: 1) Supervisor Pattern: Merkezi bir yönetici ajan gelen görevi analiz eder, hangi alt ajanın (Worker) çalışacağına karar verir ve sonuçları birleştirir. 2) Adversarial Debate (Çift Model Tartışması): İki ajan birbirinin tezini çürütmeye çalışır (ör. Biri 'Bu mimari güvenli' derken diğeri 'XSS açığı var' der). Bu yöntem mülakatlarda halüsinasyonu %90 düşüren en popüler tekniktir. 3) State Sharing: Ajanlar arası durum (State) paylaşımı genelde Redis veya merkezi bir Event-Driven mesaj kuyruğu üzerinden yönetilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Multi-Agent Topologies & Orchestration: 1) Supervisor Pattern: A centralized coordinator acts as a triage router, delegating sub-goals to specialized sub-agents and consolidating outputs. 2) Adversarial Debate Protocol: Two models critique each other's intermediate outputs in turn-based dialogue. Research proves that debate significantly reduces factual hallucination and exposes edge-case architectural vulnerabilities. 3) Shared Blackboard State: Inter-agent state is synchronized via thread-safe shared blackboards backed by Redis, PostgreSQL, or transactional memory.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Çoklu Ajan (Multi-Agent) Koordinatör Modeli
public class MultiAgentOrchestrator {
    public record AgentResponse(String role, String output) {}

    public static String runReviewPipeline(String designPrompt) {
        // 1. Coder Ajanı Tasarımı Yapar
        var coder = new AgentResponse("CODER_AGENT", "Tasarım: PostgreSQL + Redis önbelleği
kullanıldı.");
        IO.println("[" + coder.role() + "]: " + coder.output());

        // 2. Güvenlik Ajanı Denetler (Reviewer)
        var security = new AgentResponse("SECURITY_AGENT", "İnceleme: Redis şifrelemesi ve SSL zorunlu
tutulmalıdır!");
        IO.println("[" + security.role() + "]: " + security.output());

        // 3. Supervisor Birleştirir
        return "Nihai Onaylı Tasarım: PostgreSQL + SSL Korumalı Redis Önbelleği.";
    }
}

void main() {
    IO.println("=== Çoklu Ajan Koordinasyon Simülasyonu ===");
    String finalResult = MultiAgentOrchestrator.runReviewPipeline("E-ticaret sepet mimarisi");
    IO.println("Supervisor Kararı: " + finalResult);
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Maliyet Uyarısı (Token Multiplication): 3 ajanın birbiriyle 5 tur tartıştığı bir sistemde her bir mesaj tüm geçmişle birlikte katlanarak çoğalır. Tek bir kullanıcı sorgusu 50.000 token harcayabilir! Çoklu ajan sistemleri sadece yüksek değerli, kritik kararlar (finans, güvenlik denetimi) için kurulmalıdır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Token explosion warning: A 3-agent debate loop running 5 turns generates 15 compound LLM invocations with ballooning context windows. Single queries can consume 50,000+ tokens. Reserve multi-agent workflows strictly for high-stakes domains (legal audits, critical infrastructure).

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Çoklu ajan sistemleri, karmaşık görevleri uzmanlaşmış alt rollere paylaştırır.

• Supervisor mimarisi, ajanlar arasındaki koordinasyonu ve görev dağıtımını yönetir.

• Karşıt tartışma (Adversarial Debate), halüsinasyonları ve güvenlik açıklarını yakalamada rakipsizdir.

🎯 Key Takeaways for the Interview

• Multi-agent systems partition broad objectives into specialized, domain-focused sub-roles.

• The Supervisor pattern provides deterministic routing and state consolidation across workers.

• Adversarial debate between models radically suppresses hallucinations and catches flaws.

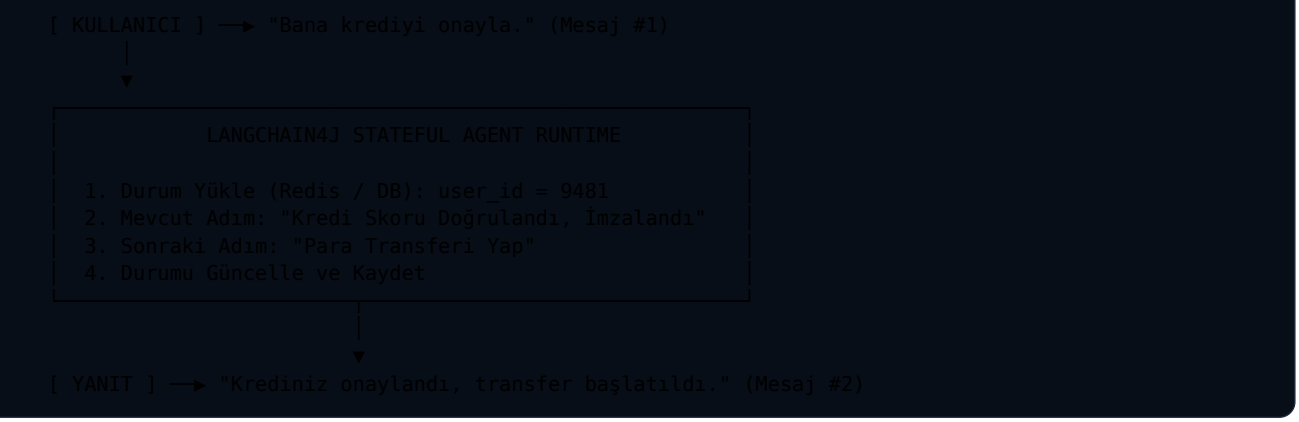
41

BÖLÜM: AI AJANLARI (AI AGENTS), OTONOM AKIŞLAR VE BELLEK MIMARISI | MODULE: AI AGENTS, AUTONOMOUS LOOPS & CONVERSATIONAL MEMORY SYSTEMS

LangChain4j ile Durumsal (Stateful) Ajan Tasarımı

Building Stateful Agents with LangChain4j: State Persistence & Resumption

🤖 AI Mimarisi & Veri Akış Şeması 🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Akıllı bir veznedarın önüne her gittiğinizde size 'Sen kimdin? Ne istiyordun?' diye sormadığını düşünün. Masasında sizin dosyanız açıktır; 'En son kimlik fotokopinizi almıştık, şimdi imzanızı atın' der (Stateful / Durumsal). LangChain4j'in durumsal ajanları böyledir: Kullanıcı tarayıcıyı kapatsa, aradan 3 gün geçse bile sistem kullanıcının nerede kaldığını hatırlar. Ajan hafızasını kaybetmez!

🇬🇧 ENGLISH (ELI5)

Imagine a bank teller who suffers complete amnesia every 30 seconds, forcing you to explain your loan application from scratch every time you speak. A stateful teller keeps your physical case file open on the desk: 'We already verified your tax returns yesterday; today we simply need your signature.' LangChain4j's stateful agents maintain persistent lifecycle states across sessions and machine restarts: an agent pauses, saves state to Redis, and resumes seamlessly days later!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Stateful Ajan Mühendisliği: 1) State Entity: Ajanın anlık değişkenlerini, tamamlanan adımlarını ve ara çıktılarını tutan Java Record modelidir. 2) Checkpointing: Her tool çağrısından sonra ajan durumu veritabanına (PostgreSQL / Redis) snapshot olarak yazılır. 3) Crash Recovery (Çökme Kurtarma): Sunucu o anda çökse veya pod yeniden başlasa bile, yeni pod son checkpoint'ten durumu yükler ve ajanı sıfırdan başlatmadan kaldığı adımdan devam ettirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Stateful Agent Architecture & Checkpointing: 1) State Machine Semantics: Models agent execution as an explicit Finite State Machine (FSM) where state transitions are triggered by tool outputs or human approvals. 2) Durable Checkpointing: After every state change, an immutable state snapshot is persisted to an ACID store (PostgreSQL or Redis JSON). 3) Crash Fault-Tolerance: If a container restarts or encounters an Out-of-Memory (OOM) error mid-task, the new pod hydrates state from the latest checkpoint, resuming execution without duplicate side-effects.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25 & LangChain4j: Durumsal Ajan Durum (State) Modeli
import java.time.Instant;
import java.util.Map;

public class AgentStateSnapshot {
    public record AgentState(
        String conversationId,
        String currentStage, // INIT, COLLECTING_INFO, AWAITING_APPROVAL, COMPLETED
        Map<String, Object> contextVariables,
        Instant lastUpdated
    ) {}

    public static AgentState advanceStage(AgentState previous, String newStage) {
        return new AgentState(
            previous.conversationId(),
            newStage,
            previous.contextVariables(),
            Instant.now()
        );
    }
}

void main() {
    var state = new AgentStateSnapshot.AgentState("conv_101", "INIT", Map.of("userId", "usr_99"),
Instant.now());
    IO.println("Ajan Başlangıç Durumu: " + state.currentStage());

    var updated = AgentStateSnapshot.advanceStage(state, "AWAITING_APPROVAL");
    IO.println("Ajan Yeni Durumu: " + updated.currentStage() + " | Güncellenme: " +
updated.lastUpdated());
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Durum Şişmesi (State Bloat): Ajanın her mesajındaki tüm verileri sonsuza kadar `contextVariables` içinde saklamak Redis'i şişirir. Görev tamamlandığında (COMPLETED) geçici durum değişkenleri temizlenmeli (TTL ile temizlik), sadece nihai özet arşivlenmelidir.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

State bloat hazard: Accumulating megabytes of unpruned intermediate tool outputs inside state objects exhausts Redis memory. Implement explicit state lifecycle management: set Redis TTLs and prune transient execution payloads upon task completion.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Durumsal (Stateful) ajanlar, uzun süren iş akışlarında oturumlar arası hafıza sağlar.
- Checkpointing, sunucu çökmelerinde ajanın kaldığı yerden devam etmesini garanti eder.
- Tamamlanan durumlar bellek şişmesini önlemek için temizlenmeli veya arşivlenmelidir.

🎯 Key Takeaways for the Interview

- Stateful agents preserve workflow context across multi-session, long-running processes.
- Checkpointing provides fault tolerance, enabling seamless resumption after server restarts.
- Completed agent states must be pruned or archived to maintain lean memory profiles.

42

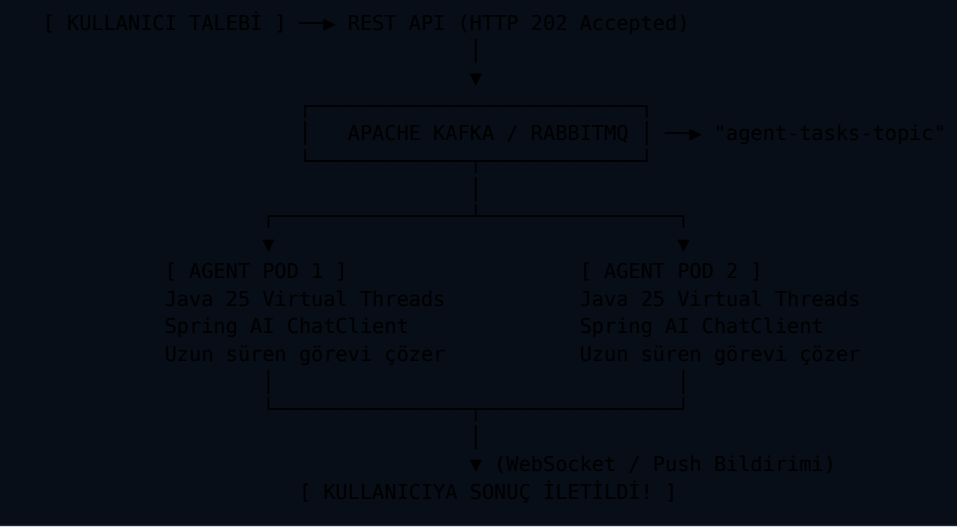
BÖLÜM: AI AJANLARI (AI AGENTS), OTONOM AKIŞLAR VE BELLEK MIMARISI | MODULE: AI AGENTS, AUTONOMOUS LOOPS & CONVERSATIONAL MEMORY SYSTEMS

Spring AI ile Dağıtık Asenkron Ajan Orkestrasyonu

Distributed Asynchronous Agent Orchestration with Spring AI & Event Streams

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda sipariş verdiğinizde garson yemeğiniz pişene kadar 30 dakika boyunca masanızın başında dikilip beklemez! Masanıza bir numara bırakır (HTTP 202 Accepted), sipariş fişini mutfağa asar (Kafka Mesajı) ve diğer masalara hizmet etmeye gider. Mutfaktaki aşçılar (Arka plan Ajanları) yemeği hazırlar ve bittiğinde zil çalar; garson yemeği masanıza getirir! Dağıtık Ajan Orkestrasyonu tam olarak budur: Saatler sürebilecek büyük yapay zeka işlerini kullanıcının ekranını kilitlemeden arka planda asenkron olarak çözmektir!

🇬🇧 ENGLISH (ELI5)

When ordering a complex banquet at a restaurant, the waiter doesn't stand frozen at your table for 45 minutes while the roast cooks. They hand you an order buzzer (HTTP 202 Accepted), pin the ticket to the kitchen queue (Kafka Event Stream), and immediately serve other patrons. Kitchen chefs (Distributed AI Agent Pods) process the orders asynchronously. When ready, the buzzer flashes! Distributed Agent Orchestration decouples heavy AI tasks from synchronous web requests!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık Ajan Mimarisi: 1) Asenkron Kabul (HTTP 202 Accepted): İstemciye hemen bir `taskId` dönülür, HTTP bağlantısı kapatılır. 2) Message Broker (Kafka / RabbitMQ): Ajan görevleri bir kuyruğa bırakılır; kuyruk arkasında çalışan onlarca Spring Boot pod'u görevleri paralel tüketir. 3) Virtual Threads ile Paralel Ajanlar: Java 25 sayesinde her bir pod içinde binlerce sanal iş parçacığı aynı anda farklı ajan döngülerini bloklanmadan yönetir. 4) Sonuç Bildirimi: Görev bittiğinde WebSocket, Webhook veya Mobil Push bildirimi ile istemci bilgilendirilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Distributed Agent Architecture & Asynchronous Topologies: 1) Decoupled Ingress: The API gateway accepts the request, returns `HTTP 202 Accepted` with a `tracking_id`, and dispatches an event. 2) Event-Driven Task Scheduling: Kafka or RabbitMQ buffers tasks, distributing workloads across scalable Spring Boot worker pools. 3) High Concurrency via Project Loom: Java 25 Virtual Threads allow a single worker pod to host thousands of concurrent, I/O-blocked agent execution loops without thread pool starvation. 4) Push Notification Delivery: Real-time status updates broadcast via WebSockets or Webhooks upon completion.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25 & Spring AI: Asenkron Görev Başlatıcı ve Virtual Thread Modeli
import java.util.UUID;
import java.util.concurrent.CompletableFuture;

public class DistributedAgentDispatcher {
    public record AgentTaskTicket(String taskId, String status) {}

    public static AgentTaskTicket submitAgentTask(String userGoal) {
        String taskId = UUID.randomUUID().toString();

        // Java 25 Virtual Threads ile arka planda asenkron ajan çalıştırma
        Thread.ofVirtual().start(() -> {
            IO.println("Arka Planda Ajan Çalışıyor (Task: " + taskId + "): " + userGoal);
            try {
                Thread.sleep(1500); // LLM ve Tool çağrıları simülasyonu
                IO.println("Ajan Görevi Başarıyla Tamamladı! (Task: " + taskId + ")");
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // İstemciye anında bilet dönülür (HTTP 202)
        return new AgentTaskTicket(taskId, "ACCEPTED");
    }
}

void main() {
    var ticket = DistributedAgentDispatcher.submitAgentTask("Tüm müşteri şikayetlerini analiz et");
    IO.println("İstemciye Dönen Yanıt: Task ID = " + ticket.taskId() + " | Durum = " +
ticket.status());
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mimari Kural: Uzun süren (10 saniyeden fazla) hiçbir ajan işini senkron bir HTTP GET/POST isteği içinde tutmayın! Bağlantı zaman aşımına (Timeout - 504 Gateway Timeout) uğrar. İşi daima asenkron kuyruğa atıp istemciye taskId dönün.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Gateway timeout antipattern: Never execute multi-step agent loops inside a synchronous HTTP request lifecycle. Ingress proxies terminate connections after 30-60 seconds. Always decouple execution via asynchronous queues and WebSockets.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Uzun süren ajan görevleri HTTP 202 Accepted ve asenkron mesaj kuyruklarıyla yönetilmelidir.
- Java 25 Virtual Threads, binlerce eşzamanlı ajan döngüsünü minimum bellek ile işletir.
- Tamamlanan görevler WebSocket veya Webhook bildirimleriyle istemciye ulaştırılır.

🎯 Key Takeaways for the Interview

- Long-running agent workflows mandate asynchronous HTTP 202 patterns and message queues.
- Java 25 Virtual Threads sustain thousands of concurrent agent loops with near-zero overhead.
- Completed task payloads are streamed back to users via WebSockets or webhook callbacks.